

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Task 7

Student Name: R Varshini

Reg. No.: A24126552115

Date: 21-09-2026

1. TITLE

Document Object Model (DOM) and Dynamic Manipulation of Content, Style and Attributes Using JavaScript

2. AIM

To explain the Document Object Model (DOM) and to demonstrate, using an HTML page and JavaScript, how the content, style and attributes of HTML elements can be changed dynamically when the user clicks a button.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what the DOM is and how an HTML page is represented as a tree of objects.
- To select an HTML element in JavaScript using **document.getElementById()**.
- To change the content of an element using the **innerHTML** property.
- To change the style of an element using the **style** property (color and font size).
- To change the attributes of an element using **setAttribute()** (src and alt of an image).
- To run JavaScript in response to a user action using the **onclick** event.

4. THEORY

The **Document Object Model (DOM)** is a programming interface for HTML documents. When a browser loads a web page, it reads the HTML and builds a tree of objects, called *nodes*, in memory. Every tag, attribute and piece of text in the page becomes a node in this tree. JavaScript can use the DOM to find any node and to add, remove or modify it, and the browser updates the page immediately without reloading. This is what makes a web page dynamic.

- **document** is the root object of the DOM; through it JavaScript reaches every element of the page.
- **Node types:** element nodes (tags such as h1, p, img), attribute nodes (id, src, alt), text nodes (the text inside an element) and comment nodes.
- **document.getElementById(id)** returns the element having the given id, or **null** if no such element exists.
- **Content:** `element.innerHTML = value` replaces everything inside the element with the new text or HTML.

- **Style:** `element.style.property = value` sets an inline CSS property; CSS names are written in camelCase (font-size becomes `fontSize`).
- **Attributes:** `element.setAttribute(name, value)` adds or updates an attribute and `getAttribute(name)` reads it.
- **Events:** the `onclick` attribute calls a JavaScript function when the element is clicked, so the DOM is changed only when the user acts.

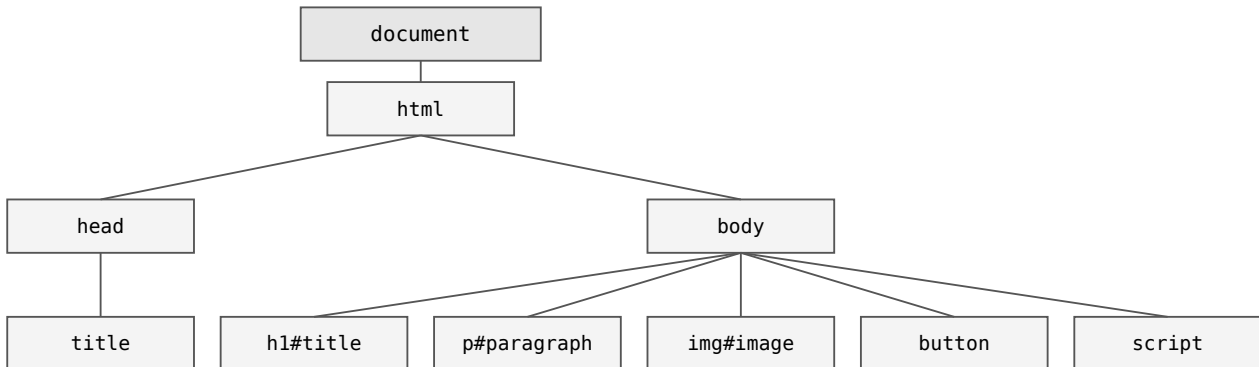


Fig. 1: DOM tree (element nodes) built by the browser for `pbm3.html`

What is changed	DOM property / method	Used in this program
Content	<code>innerHTML</code>	<code>title.innerHTML = "DOM Manipulation in Action!"</code>
Style	<code>style.color</code> , <code>style.fontSize</code>	<code>paragraph.style.color = "blue"</code>
Attributes	<code>setAttribute()</code>	<code>image.setAttribute("src", "new.jpg")</code>

Flow: Page loads → browser builds the DOM tree → user clicks the button → `onclick` calls `changeContent()` → `getElementById()` selects the elements → content, style and attributes are modified → browser re-renders the page

5. ALGORITHM / PROCEDURE

1. Start. Create an HTML file **pbm3.html** containing a heading (`id="title"`), a paragraph (`id="paragraph"`), an image (`id="image"`, `src="old.jpg"`, `alt="Old Image"`) and a button.
2. Attach the event handler `onclick="changeContent()"` to the button.
3. Write a **changeContent()** function inside the `<script>` tag.
4. Select the heading using `getElementById("title")` and set its `innerHTML` to "DOM Manipulation in Action!".
5. Select the paragraph using `getElementById("paragraph")` and set its `innerHTML` to "The content has been updated dynamically.".
6. Change the style of the paragraph: set `style.color` to "blue" and `style.fontSize` to "20px".
7. Select the image using `getElementById("image")` and use `setAttribute()` to change `src` to "new.jpg" and `alt` to "Updated Image".
8. Open `pbm3.html` in a web browser and check that the original page is displayed.
9. Click the **Click Me** button and observe that the content, style and attributes change without reloading the page.
10. Verify the result for each test case (and with the browser console) and stop.

6. PROGRAM

pbm3.html

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM Example</title>
</head>
<body>
  <!-- Original elements -->
  <h1 id="title">Hello World!</h1>
  <p id="paragraph">This is a paragraph.</p>
  

  <!-- Button to trigger changes -->
  <button onclick="changeContent()">Click Me</button>

  <script>
    function changeContent() {
      // Change content
      document.getElementById("title").innerHTML = "DOM Manipulation in Action!";
      document.getElementById("paragraph").innerHTML = "The content has been updated dynamically.";

      // Change style
      document.getElementById("paragraph").style.color = "blue";
      document.getElementById("paragraph").style.fontSize = "20px";

      // Change attributes
      document.getElementById("image").setAttribute("src", "new.jpg");
      document.getElementById("image").setAttribute("alt", "Updated Image");
    }
  </script>
</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
<h1 id="title">	Heading element. The id lets JavaScript find it in the DOM tree.
<p id="paragraph">	Paragraph whose text and style are changed by the script.
	Image element; its src and alt attributes are changed by the script.
onclick="changeContent()"	Event attribute: calls the function changeContent() when the button is clicked.
<script> ... </script>	Contains the JavaScript that manipulates the DOM.
function changeContent()	Function that performs all the changes when it is called.
document.getElementById("title")	Searches the DOM tree for the element with id "title" and returns it as an object.
.innerHTML = "..."	Replaces the content inside the element with the new text.
.style.color = "blue"	Sets the inline CSS colour of the paragraph text to blue.
.style.fontSize = "20px"	Sets the font size of the paragraph to 20 pixels (font-size in camelCase).
.setAttribute("src", "new.jpg")	Changes the src attribute, so the image now points to new.jpg.
.setAttribute("alt", "Updated Image")	Changes the alt text of the image to "Updated Image".

8. TEST CASES AND EXECUTION DATA

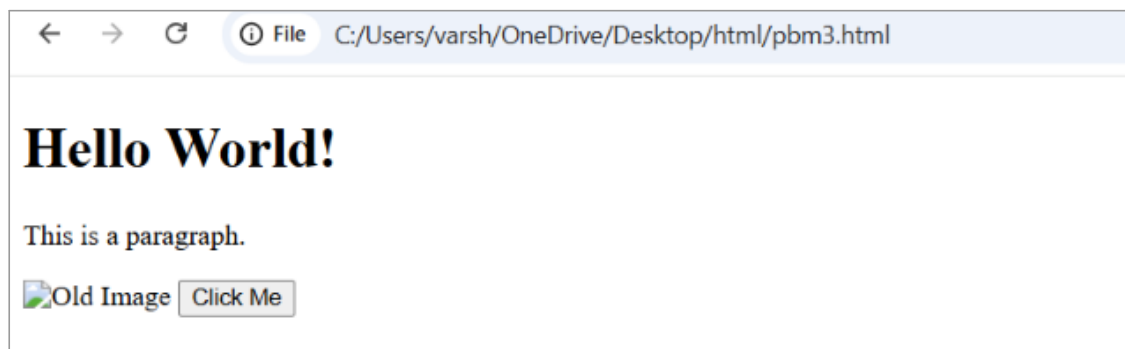
Each test case was executed by opening `pbm3.html` in a Chromium-based web browser (Google Chrome) and clicking the button where required. The observed results were recorded exactly as displayed.

Test Case	Scenario / Action	Expected Result	Actual Result	Status
TC-01	Open <code>pbm3.html</code> (before clicking the button)	Heading "Hello World!"; paragraph "This is a paragraph." in default black; image with alt text "Old Image"; button visible.	Same as expected.	Pass
TC-02	Click the "Click Me" button once	Heading changes to "DOM Manipulation in Action!"; paragraph text is updated and shown in blue, 20px; image alt becomes "Updated Image".	Same as expected.	Pass
TC-03	Keep <code>old.jpg</code> and <code>new.jpg</code> in the same folder; click the button	Old image is shown before the click; new image is shown after the click.	Same as expected.	Pass
TC-04	Click the button three times	Same result as a single click; no repeated text and no error.	Same as expected.	Pass
TC-05	Check the values in the browser console after the click	innerHTML values updated; <code>style.color = "blue"; style.fontSize = "20px"; src = "new.jpg"; alt = "Updated Image".</code>	Same values returned by the console.	Pass
TC-06	Change the id of the image to "img1" (wrong id) and click the button	<code>TypeError: getElementById("image") returns null.</code> Heading and paragraph (changed before the error) stay updated; image is unchanged.	Same as expected; error shown in console.	Pass

Note: `old.jpg` and `new.jpg` are not present in the folder in TC-01, TC-02, TC-04 and TC-06, so the browser shows the alt text of the image. This itself shows that the alt attribute changed from "Old Image" to "Updated Image".

9. OUTPUT

Output 1: TC-01 – page before clicking the button



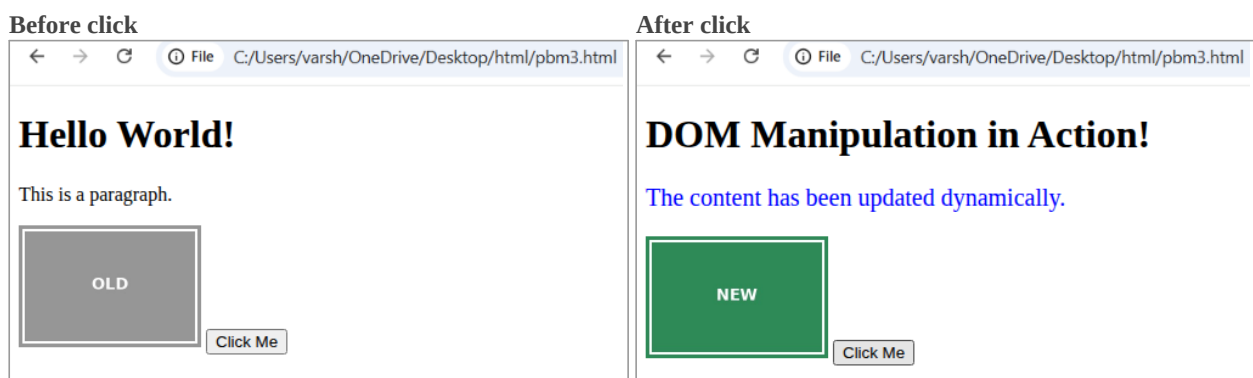
The original page shows the heading, the paragraph, the image (alt text "Old Image") and the **Click Me** button.

Output 2: TC-02 – page after clicking the button



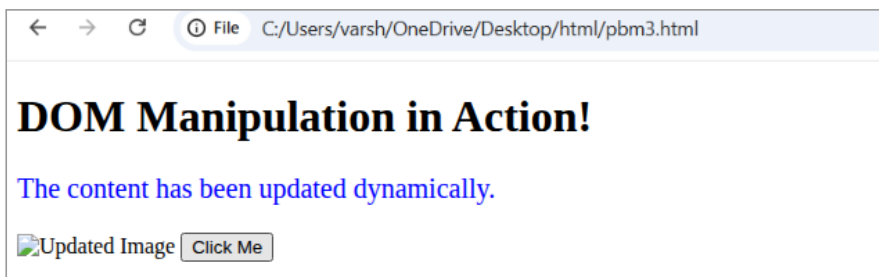
The heading and paragraph text are changed, the paragraph is now blue and larger (20px), and the image alt text is now "Updated Image".

Output 3: TC-03 – image attribute change (old.jpg and new.jpg present)



Changing the src attribute makes the browser load new.jpg in place of old.jpg.

Output 4: TC-04 – button clicked three times



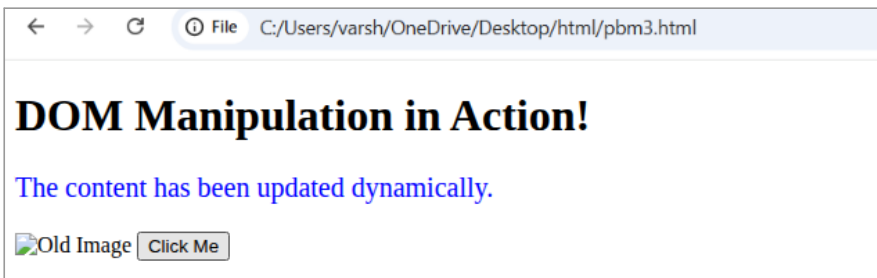
The page shows the same final state as after one click; repeated clicks only set the same values again.

Output 5: TC-05 – values checked in the browser console after the click

```
Elements Console Sources Network
> document.getElementById("title").innerHTML
< 'DOM Manipulation in Action!'
> document.getElementById("paragraph").innerHTML
< 'The content has been updated dynamically.'
> document.getElementById("paragraph").style.color
< 'blue'
> document.getElementById("paragraph").style.fontSize
< '20px'
> document.getElementById("image").getAttribute("src")
< 'new.jpg'
> document.getElementById("image").getAttribute("alt")
< 'Updated Image'
```

The console confirms the new content, colour, font size and the changed src and alt attributes.

Output 6: TC-06 – wrong element id (img1)



```
Elements Console Sources Network
* Uncaught TypeError: Cannot read properties of null (reading 'setAttribute')
    at changeContent (pbm3.html:26:39)
    at HTMLButtonElement.onclick (pbm3.html:13:37)
> document.getElementById("title").innerHTML
< 'DOM Manipulation in Action!'
> document.getElementById("img1").getAttribute("alt")
< 'Old Image'
```

getElementById("image") returns null, so setAttribute() throws a TypeError. The heading and paragraph were already changed before the error, but the image is unchanged.

10. RESULT

Thus, the Document Object Model (DOM) was studied and a web page was developed in which JavaScript dynamically changes the content, style and attributes of HTML elements when a button is clicked. The program was verified using multiple test cases.